

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Viện Trí Tuệ Nhân Tạo



PHƯƠNG PHÁP LUẬN LẬP TRÌNH

BÁO CÁO
LEVEL_1

Sinh viên thực hiện

Đỗ Minh Trí

Mã sinh viên: 25023059

Hà Nội, năm học 2025 – 2026

MỤC LỤC

I. Overview 1

II. Chi tiết cài đặt 2

III. Kết quả thực nghiệm 8

IV. Nhận xét và tổng kết bài làm 10

V. Nguồn tham khảo..... 11

VI. Ghi chú sử dụng AI 12

I. Overview

Phần bài làm đã thành công cài đặt tổng cộng **26 chức năng**:

Module Game Interaction:

- Đọc và kiểm tra tính hợp lệ của dữ liệu nhập từ bàn phím (`validateInput`, `getInput`).
- Lựa chọn cấu hình trò chơi (`selectSize`, `selectGoal`, `selectGameMode`, `selectBotLevel`).
- Xử lý việc nhận tọa độ nước đi từ người chơi (`getPlayerMove`).

Module Game Renderer:

- Thiết kế giao diện bàn cờ trực quan với đầy đủ chỉ số hàng, cột và hiển thị các ký hiệu ``X'`, `O'`, `-' (displayBoard).`
- Hiển thị tiêu đề và các lựa chọn cấu hình trò chơi (`showSelectMenu`).
- Hệ thống thông báo trạng thái lượt đi, nước đi và kết quả trận đấu (`showPlayer`, `showMove`, `showInvalidMove`).
- Cập nhật kết quả trận đấu (`showResult`, `printResult`).

Module Game Logic:

- Quản lý bàn cờ (`initBoard`, `isValidMove`, `isEmptyHead`, `makeMove`).
- Cài đặt thuật toán kiểm tra thắng cuộc đa hướng (ngang, dọc, chéo) tích hợp luật chặn hai đầu (`checkWin`).
- Xử lý logic cập nhật bàn cờ và kiểm tra trạng thái hòa (`checkDraw`).

Module Game Engine:

- Thiết lập vòng đời trò chơi từ khâu khởi tạo đến khi kết thúc (`startGame`, `playGame`, `endGame`).

Bot Module:

- EASY: Chọn ngẫu nhiên ô trống (`random_pick`).
- MEDIUM: Tìm nước thắng ngay hoặc chặn đối thủ thắng; nếu không có thì chọn

nước đi đơn giản hoặc nước đi trung tâm (`simple_heuristic`).

- **HARD:** Mở rộng từ MEDIUM bằng cách đánh giá heuristic các nước đi (dựa trên độ dài chuỗi, số đầu mở) cho cả tấn công và phòng thủ, từ đó chọn nước đi tối ưu cục bộ (`hard_level`).

II. Chi tiết cài đặt

1. Game Interaction: Bạn đã xử lý việc xác thực dữ liệu đầu vào (input validation) như thế nào? Trong trường hợp người dùng nhập ký tự không phải số hoặc tọa độ ngoài phạm vi, chương trình sẽ phản hồi ra sao để đảm bảo không bị treo (crash)?

Để đảm bảo chương trình không bị crash, hai lớp bảo vệ chồng nhau đã được sử dụng. **Lớp thứ nhất** là hàm `getInput()`. Thay vì đọc chuỗi rồi parse thủ công, nó dùng `std::cin >>` trực tiếp. Khi người dùng nhập ký tự không phải số (ví dụ "abc"), `std::cin` tự động chuyển sang trạng thái fail. Lúc đó hàm gọi `std::cin.clear()` để reset trạng thái và `std::cin.ignore(...)` để xóa sạch buffer, sau đó trả về `false`:

```
bool getInput(int *val)
{
    if (std::cin >> *val)
        return true;
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    return false;
}
```

Hình 1. Mô tả hàm `getInput()`

Lớp thứ hai là vòng lặp `while (true)` trong `startGame()`. Mỗi lần `selectSize()`, `selectGoal()`, `selectGameMode()` trả về `false` thì chương trình in lại menu và hỏi tiếp, không bao giờ tiến sang bước tiếp theo khi input chưa hợp lệ:

```
while (true)
{
    showSelectMenu(SelectType::SIZE_UI);
    if (selectSize(&gameSetup.size))
        break;
}
```

Hình 2. Vòng lặp `while (true)` trong `startGame()`

Với tọa độ ngoài phạm vi, hàm `isValidMove()` thực hiện kiểm tra biên (boundary check) để đảm bảo an toàn bộ nhớ. Việc kiểm tra này giúp ngăn chặn triệt để lỗi *out-of-bounds* và tránh các hành vi không xác định (*undefined behavior*):

```
bool isValidMove(const char board[][BOARD_N_MAX],
                const int size,
                const int row,
                const int col)
{
    return (row >= 0 && row < size && col >= 0 && col < size && board[row][col] == '-');
}
```

Hình 3. Mô tả hàm `isValidMove()`

2. Game Renderer: Hãy mô tả định dạng hiển thị bàn chơi mà bạn đã cài đặt. Làm thế nào để người chơi dễ dàng phân biệt giữa các hàng, cột và ký hiệu của hai bên (X/O) trên console?

Bàn chơi được hiển thị dưới dạng lưới với các chỉ số hàng và cột (0, 1, 2...) bao quanh để người chơi dễ dàng định vị. Ký hiệu của hai bên (**X** và **O**) được hiển thị rõ nét trên nền các ô trống (ký hiệu là -), giúp phân biệt luồng đi và trạng thái bàn cờ một cách trực quan trên giao diện console.

```

      0 1 2 3 4
      -----
0 | O X - X X
1 | - - - - X
2 | - - X O X
3 | - - O - -
4 | - O - - O
```

Hình 4. Giao diện bàn cờ

3. Game Logic:

- Viết mã giả (pseudo-code) cho thuật toán `checkWin(...)`: Bạn đã tối ưu việc quét các hướng (ngang, dọc, chéo) như thế nào để đảm bảo hiệu suất khi kích thước bàn cờ lớn?
- Mô tả logic của hàm `checkDraw(...)`: Điều kiện chính xác để xác định một trận hòa là gì?

Mã giả thuật toán `checkWin`: Thay vì duyệt toàn bộ bàn cờ, tối ưu bằng cách chỉ quét theo 4 hướng (ngang, dọc, chéo chính, chéo phụ) xuất phát từ vị trí nước đi cuối cùng.

```

1 // 4 hướng: ngang, dọc, chéo chính, chéo phụ
2 dx = {0, 1, 1, 1}
3 dy = {1, 0, 1, -1}
4
5 for dir = 0 -> 3:
6     for i = 0 -> size - 1:
7         for j = 0 -> size - 1:
8             if board[i][j] != symbol: continue
9             count = 1
10            ni = i + dx[dir]
11            nj = j + dy[dir]
12            while valid(ni, nj) and board[ni][nj] == symbol:
13                count += 1
14                ni += dx[dir]
15                nj += dy[dir]
16            if count < goal: continue
17            headOpen = check cell before (i, j)
18            tailOpen = check cell after (ni, nj)
19            if rule == NONE: return true
20            if rule == OPEN_ONE and (head or tail): return true
21            if rule == OPEN_TWO and (head and tail): return true
22 return false

```

Thay vì 4 vòng lặp riêng biệt cho 4 hướng, phiên bản dùng mảng hướng `dx[]`, `dy[]` và gộp vào một vòng lặp duy nhất. Điều này giảm số lần duyệt từ $4 \times O(N^2 \times \text{goal})$ xuống còn $1 \times O(N^2 \times \text{goal})$, tiết kiệm cache và code ngắn gọn hơn. Ngoài ra, vòng đếm dừng ngay khi gặp ký tự khác, tránh duyệt thừa.

Mô tả logic của hàm `checkDraw(...)`: Điều kiện hòa: Không còn ô trống nào trên bảng.

```
bool checkDraw(char board[][BOARD_N_MAX],
               const int size)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            if (board[i][j] == '-')
                return false;
        }
    }
    return true;
}
```

Hình 5. Mô tả hàm `checkDraw()`

4. Game Engine: Hãy giải thích cách bạn điều phối lượt đi giữa Người và Bot trong vòng lặp chính. Làm thế nào để Engine nhận biết được trò chơi đã kết thúc để gọi hàm `endGame`?

Engine nhận biết trò chơi kết thúc qua hai điều kiện `checkWin` và `checkDraw` đặt ngay sau mỗi nước đi. Khi một trong hai trả về `true`, vòng lặp `break` ngay lập tức, `GameResult` được điền đầy đủ (`winner`, `isBot`, `turns`) rồi trả về cho `main().main()` sau đó gọi `endGame()` để hiển thị kết quả.

5. Sử dụng Game Helper: Bạn đã sử dụng hàm `measureExecutionTime` như thế nào để đánh giá hiệu năng của Bot? Hãy trích dẫn một kết quả đo thực tế từ log và nhận xét về sự chênh lệch thời gian xử lý giữa các cấp độ Bot (nếu có).

Trong `playGame()`, mỗi khi đến lượt bot, nước đi được wrap trong hàm đo thời gian:

```

while (true)
{
    pII move = measureExecutionTime("botMove", [&]()
    { return botMove(gameSetup.board, gameSetup.size, gameSetup.goal,
        symbols[currentPlayer], gameSetup.levels[currentPlayer]); }, TIME_ENABLED);
}

```

Hình 6. Mô tả hàm `measureExecutionTime()`

Hàm này dùng `std::chrono::high_resolution_clock` để đo chính xác đến nano-giây. Kết quả log thực tế:

game.log — Bot execution time

```

=== EASY Bot (5x5, goal=3) ===
[DEBUG] - [2026-04-20 05:35:56] execution time of [botMove()] =
6.527e-06s
[DEBUG] - [2026-04-20 05:35:57] execution time of [botMove()] =
4.683e-06s
[DEBUG] - [2026-04-20 05:35:59] execution time of [botMove()] =
3.725e-06s
[DEBUG] - [2026-04-20 05:36:00] execution time of [botMove()] =
3.941e-06s
[DEBUG] - [2026-04-20 05:36:02] execution time of [botMove()] =
3.997e-06s

=== MEDIUM Bot (5x5, goal=3) ===
[DEBUG] - [2026-04-20 05:36:17] execution time of [botMove()] =
9.435e-06s
[DEBUG] - [2026-04-20 05:36:18] execution time of [botMove()] =
1.5612e-05s
[DEBUG] - [2026-04-20 05:36:21] execution time of [botMove()] =
3.5125e-05s
[DEBUG] - [2026-04-20 05:36:23] execution time of [botMove()] =
1.7333e-05s
[DEBUG] - [2026-04-20 05:36:24] execution time of [botMove()] =
1.0344e-05s

=== HARD Bot (5x5, goal=3) ===
[DEBUG] - [2026-04-20 05:36:36] execution time of [botMove()] =
1.2283e-05s
[DEBUG] - [2026-04-20 05:36:38] execution time of [botMove()] =
1.3273e-05s
[DEBUG] - [2026-04-20 05:36:39] execution time of [botMove()] =
1.5019e-05s
[DEBUG] - [2026-04-20 05:36:44] execution time of [botMove()] =

```



```
9.089e-06s
```

```
[DEBUG] - [2026-04-20 05:36:48] execution time of [botMove()] =
```

```
1.8491e-05s
```

- **EASY (3–7 μ s):** `random_pick()` chỉ duyệt một lần toàn bộ ô trống, đẩy vào vector, rồi lấy chỉ số ngẫu nhiên. Không có logic phán đoán nào, nên thời gian gần như tuyến tính và dao động cực nhỏ.
- **MEDIUM (7–37 μ s):** `simple_heuristic()` thực hiện hai vòng duyệt toàn bộ bảng: lần 1 thử đặt quân bot, lần 2 thử quân đối thủ, mỗi lần đều gọi `checkWin()`.
- **HARD (6–18 μ s):** Duyệt bảng một lần, tính `scoreCell()` cho từng ô dựa trên chiều dài chuỗi và đầu mở, rồi chọn ô có tổng điểm cao nhất.

Bot Module – Chi tiết từng cấp độ:

- **EASY – `random_pick`:** Duyệt toàn bộ bàn cờ để tìm các ô trống '-' và đưa vào danh sách `empty_cells`. Sử dụng `std::uniform_int_distribution` để chọn chỉ số ngẫu nhiên. AI không tính toán nước đi chiến thắng.
- **MEDIUM – `simple_heuristic`:** Bot gồm 3 tầng ưu tiên:
 - **Tầng 1 – Thắng ngay:** Đặt thử `botSymbol` vào từng ô trống, nếu `checkWin(OPEN_TWO) = true` thì trả về ngay.
 - **Tầng 2 – Chặn đối thủ:** Đặt thử `playerSymbol` vào từng ô trống, nếu `checkWin(OPEN_TWO) = true` thì trả về ngay.
 - **Tầng 3 – Trung tâm hoặc random:** Chọn ô trung tâm (`size/2, size/2`) nếu trống, ngược lại dùng `random_pick()`.
- **HARD – Đánh giá điểm số:** Bot Hard dùng 3 hàm chính:
 - **Hàm 1:** Đếm số quân liên tiếp từ (x, y) theo một hướng.
 - **Hàm 2:** Tính chuỗi và đầu mút theo 4 hướng rồi tra bảng điểm.
 - **Hàm 3:**
 - * **Bước 1:** Kiểm tra thắng ngay / chặn thắng ngay.
 - * **Bước 2:** Tính `totalScore = attackScore  $\times$  1.2 + defendScore`; chọn ô điểm cao nhất.
 - * **Bước 3:** Fallback về trung tâm hoặc random.

III. Kết quả thực nghiệm

Giao diện tương tác (Menu thiết lập, bàn chơi, thông báo nước đi):

```
>----- Tic-tac-toe [Console v0.4.25023059] -----<

Enter the board size (NxN, 3 <= N <= 12): 5
Enter the goal (3 - 5, goal <= size): 3
Select Your Game Mode:
(1) PvP | (2) PvE | (3) EvE3
Bot 1 - Select Bot Level:(1) EASY | (2) MEDIUM | (3) HARD3
Bot 2 - Select Bot Level:(1) EASY | (2) MEDIUM | (3) HARD1
```

Hình 7. Menu thiết lập trò chơi

```

      0 1 2 3 4
      -----
0 | - - - - -
1 | - X X - -
2 | - - 0 0 -
3 | - 0 - X -
4 | - - - - -

Player 1's turn

```

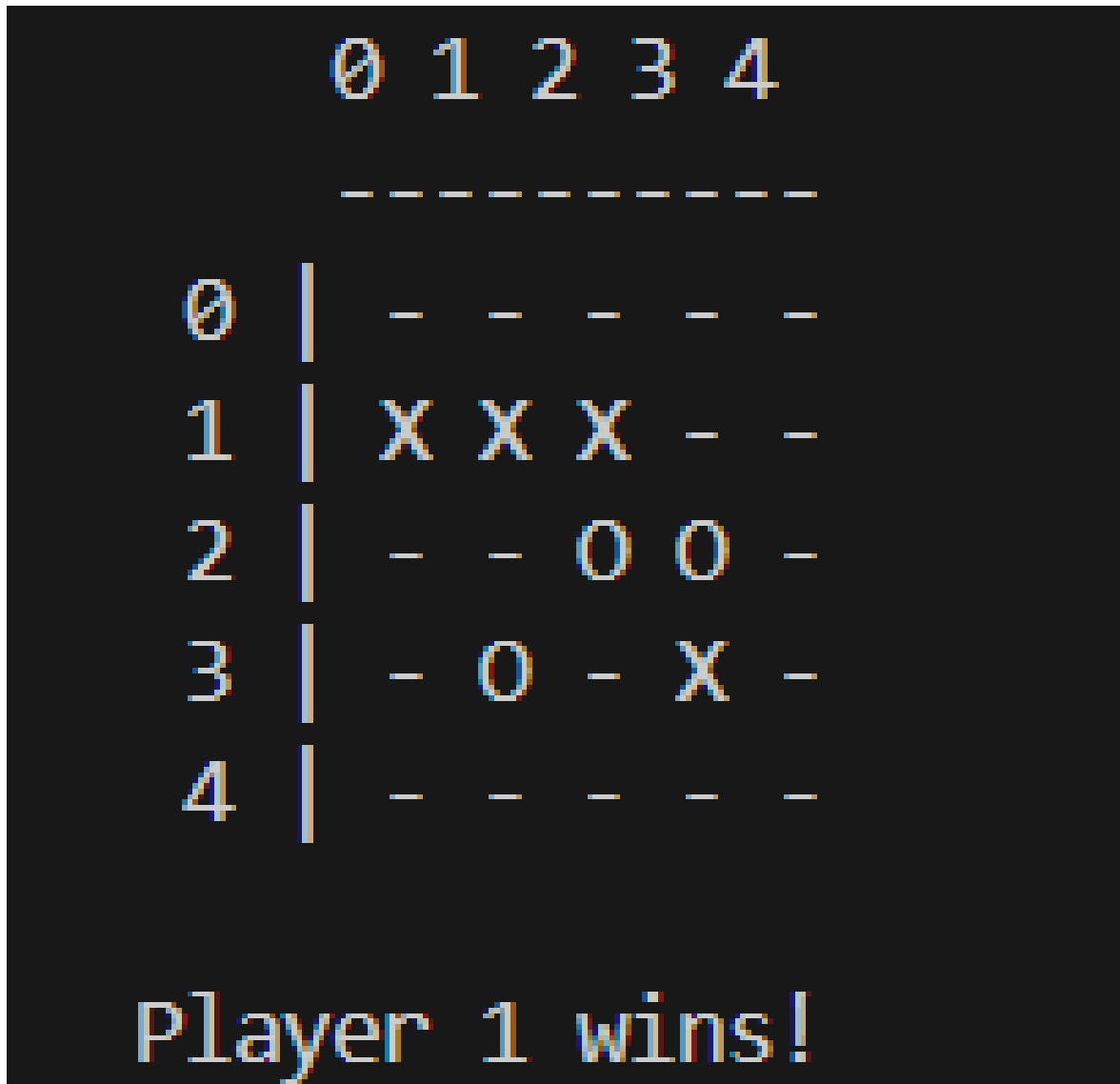
```

      0 1 2 3 4
      -----
0 | - - 0 - -
1 | - X - - -
2 | - X - - -
3 | - - 0 - -
4 | - - - - -

Bot (Player 1) is thinking...

```

Hình 8. Giao diện trong trận đấu



Hình 9. Thông báo kết thúc trận đấu

Báo cáo Grader – Kết quả chạy từ `grader.py` (số lượng test pass, tỷ lệ, accuracy):

```
PS C:\Cpp> python grader.py --target game
Grading start for game...
-----
[1/5] Testing: input1.txt... PASSED
[2/5] Testing: input2.txt... PASSED
[3/5] Testing: input3.txt... PASSED
[4/5] Testing: input4.txt... PASSED
[5/5] Testing: input5.txt... PASSED
-----
GRADING OVERALL:
- Total testcase: 5
- Corrects: 5
- Accuracy: 100.00%
Good job! You've passed all testcase.
```

Hình 10. Kết quả chạy Grader

Trích dẫn Log – Quản lý luồng dữ liệu và đo thời gian thực thi:

```
Tic-tac-toe Game (Version: 0.4.25023059)
-----
[INFO] - [2026-04-20 10:35:17] GameLogger initialized!
[INFO] - [2026-04-20 10:35:17] GameInteraction initialized!
[INFO] - [2026-04-20 10:35:25] GameEngine initialized!
[DEBUG] - [2026-04-20 10:35:26] execution time of [botMove()] = 1.43e-05s
[DEBUG] - [2026-04-20 10:35:28] execution time of [botMove()] = 1.6e-05s
[DEBUG] - [2026-04-20 10:35:29] execution time of [botMove()] = 2.5e-05s
[DEBUG] - [2026-04-20 10:35:31] execution time of [botMove()] = 6.5e-06s
[DEBUG] - [2026-04-20 10:35:32] execution time of [botMove()] = 1.76e-05s
[DEBUG] - [2026-04-20 10:35:34] execution time of [botMove()] = 9.3e-06s
[DEBUG] - [2026-04-20 10:35:35] execution time of [botMove()] = 7e-06s
[DEBUG] - [2026-04-20 10:35:37] execution time of [botMove()] = 9.9e-06s
[DEBUG] - [2026-04-20 10:35:38] execution time of [botMove()] = 2.39e-05s
[DEBUG] - [2026-04-20 10:35:40] execution time of [botMove()] = 5.44e-05s
[DEBUG] - [2026-04-20 10:35:41] execution time of [botMove()] = 4.4e-06s
[INFO] - [2026-04-20 10:35:41] GameEngine playing done!
[INFO] - [2026-04-20 10:35:41] GameEngine show endgame done!
[INFO] - [2026-04-20 10:35:41] GameInteraction closed!
[INFO] - [2026-04-20 10:35:41] GameLogger closing . . .
```

Hình 11. Log tiêu biểu từ game.log

IV. Nhận xét và tổng kết bài làm

1. Về phong cách lập trình: Theo trải nghiệm của bạn, việc sử dụng phong cách lập trình thủ tục (Procedural Programming) có điểm mạnh và điểm yếu gì trong việc quản lý luồng trò chơi? Liệu cách tiếp cận này có dễ bảo trì khi trò chơi trở nên phức tạp hơn không?

Luồng chương trình rõ ràng, dễ đọc theo thứ tự tuyến tính từ `startGame()` → `playGame()` → `endGame()`. Với người mới, không cần hiểu về class hay kế thừa vẫn theo dõi được logic. Tuy nhiên, khi thêm tính năng mới (ví dụ: chế độ chơi mạng, lưu lịch sử ván), các hàm bắt đầu phụ thuộc lẫn nhau nhiều hơn, dữ liệu như `board`, `size`, `goal` phải truyền qua lại liên tục, rất dễ gây lỗi và khó refactor. Về lâu dài, không dễ bảo trì – một thay đổi nhỏ trong `checkWin` có thể ảnh hưởng đến cả `simple_heuristic` lẫn `hard_level`.

2. Về giao diện người dùng: Làm việc với Terminal Interface (dòng lệnh) mang lại những điều thú vị và khó khăn gì so với các ứng dụng có giao diện đồ họa (GUI) thông thường?

Buộc phải suy nghĩ về UX theo cách khác, mỗi ký tự in ra đều có chủ đích. Việc dùng màu ANSI (`\033[31m...`) để tô màu bảng hoặc thông báo lỗi tạo cảm giác mới

mẻ cho terminal. Mặt khác, xử lý input phức tạp hơn nhiều so với GUI khi phải tự validate từng trường hợp (nhập chữ thay số, nhập âm, nhập ngoài range...). Ngoài ra, không có event-driven model nên mọi thứ phải tuần tự, khó tạo hiệu ứng động hay phản hồi realtime.

3. Về tổ chức mã nguồn: Việc tập trung toàn bộ mã nguồn vào một file (monolithic file) dù đã có chú thích rõ ràng có thực sự hiệu quả cho việc phát triển lâu dài? Theo bạn, có cách tổ chức nào khác giúp chương trình chuyên nghiệp và dễ quản lý hơn không?

Dù có comment rõ ràng và chia section bằng các khối `/* -- */`, một file 2600+ dòng vẫn không thực sự hiệu quả khi dự án lớn lên, gây tìm kiếm chậm, merge conflict khi làm nhóm, và khó test từng module riêng lẻ.

4. Bài học rút ra: Khó khăn lớn nhất bạn đã vượt qua là gì (ví dụ: lỗi logic `checkWin`, xử lý nhập liệu, hay tối ưu thuật toán Bot)?

Khó khăn lớn nhất là `checkWin` với `EndRule` – đặc biệt là `OPEN_TWO`, vì không chỉ đơn giản đếm goal ký tự liên tiếp mà còn phải kiểm tra cả hai đầu chuỗi có trống không, xử lý đúng biên bảng, và đảm bảo logic nhất quán khi dùng trong cả `simple_heuristic` lẫn `hard_level`. Một lỗi nhỏ ở đây kéo theo bot ra quyết định sai hoàn toàn.

V. Nguồn tham khảo

Project này đã được hỗ trợ thực hiện dựa vào sự trợ giúp của nhiều nguồn trên Internet:

- Các tài liệu lý thuyết và thực hành của môn Phương pháp luận lập trình.
- Video YouTube hỗ trợ hiểu bản chất và cách sử dụng con trỏ, tham chiếu:
 - Tất Tần Tật Về Con Trỏ C++ Trong 10 Phút
 - Chinh phục Con Trỏ (Pointer) | Vũ Nguyễn Coder
- Tham khảo cơ chế hoạt động Bot Easy:
 - C++ TIC TAC TOE game for beginners
- Thuật toán `checkWin()` và ý tưởng cho Bot Hard:
 - dev.to – A Gomoku AI with Minimax & Alpha-Beta Pruning

- Học những bước đầu sử dụng công cụ $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$:
 - Playlist LaTeX Tutorials – Dr. Trefor Bazett

Một số nguồn có thể không được sử dụng trực tiếp trong project mà mang tính chất tham khảo ý tưởng, giúp người học hiểu hơn về chương trình mình đang thực thi.

VI. Ghi chú sử dụng AI

Để hoàn thiện project này, hai công cụ AI đã được sử dụng là **Gemini** và **Claude**:

- **Gemini** chủ yếu hỗ trợ trả lời các câu hỏi (con trỏ, tham chiếu, giải thích yêu cầu...). Ngoài ra, bài trình bày sử dụng $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ được Gemini hỗ trợ rất nhiều do người làm mới bắt đầu tiếp xúc với ngôn ngữ này (sử dụng thư viện, format, chỉnh sửa).
- **Claude** chủ yếu giúp đỡ trong việc đưa ra ý tưởng và giải thích thuật toán, gợi ý cách triển khai, kiểm tra lỗi toàn diện, tối ưu hóa. Đồng thời, Claude hỗ trợ tạo một cấu trúc $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ có sẵn giúp trình bày báo cáo khoa học, đẹp mắt hơn.

Sản phẩm này đã được AI trực tiếp hỗ trợ rất nhiều (khoảng **30–40%**). Đóng góp lớn nhất AI mang lại là khả năng thúc đẩy học tập, tiếp cận đối với những kiến thức và công cụ mới.